

# RSP Backend — API Reference (for building a new UI)

Full picture of the backend so a new frontend can be built against it without guessing. The **authoritative, machine-readable** version is `openapi.json` (FastAPI's auto-generated OpenAPI 3 spec) — feed that to design/codegen tools. This file is the readable summary.

## Conventions (read first)

- **Base URL:** every endpoint is under `/api/v1` (except `GET /health`).
- **Dev setup:** frontend (Vite, :5173) proxies `/api/*` → backend ( :8000 ). So the frontend calls relative paths like `/api/v1/events` . ( `vite.config.js` does the proxy.)
- **Auth:** JWT Bearer. Get a token from `POST /auth/login` , then send `Authorization: Bearer <token>` on every other request. Only `login` , `register` , `health` are public.
- **Format:** JSON in/out. **Datetimes** are **ISO-8601 UTC** (e.g. `2026-06-10T10:00:00Z` ). **IDs** are **UUID strings**. Enum values in JSON are the **lowercase** values listed below.
- **Errors:** non-2xx returns `{ "detail": "message" }` . Common: `400` validation, `401` bad/missing token, `403` wrong role, `404` not found, `409` conflict (room taken / student clash / FSM).
- **Roles:** `admin` , `professor` , `staff` , `viewer` . "Admin-only" endpoints are marked  .

## 1. Auth

| Method | Path                        | Auth   | Body                                        | Returns                                                   |
|--------|-----------------------------|--------|---------------------------------------------|-----------------------------------------------------------|
| POST   | <code>/auth/login</code>    | public | <code>{ email, password }</code>            | <code>{ access_token, ...user }</code> (token + the user) |
| POST   | <code>/auth/register</code> | public | <code>{ email, full_name, password }</code> | the created user                                          |
| GET    | <code>/auth/me</code>       | user   | —                                           | the current user                                          |

User object: `{ id, email, full_name, role, is_active, created_at }` .

## 2. Users & Notifications

| Method | Path                                          | Auth                                                                                    | Body                                  | Returns             |
|--------|-----------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------|---------------------|
| GET    | /users                                        |  admin | —                                     | User[]              |
| GET    | /users/{user_id}                              |  admin | —                                     | User                |
| PATCH  | /users/{user_id}                              |  admin | {ufl_name?,<br>role?,<br>is_active? } | User                |
| GET    | /users/me/notifications?<br>unread_only=false | user                                                                                    | —                                     | Notification[]      |
| POST   | /users/me/notifications/read                  | user                                                                                    | —                                     | 204 (mark all read) |

**Notification:** { id, recipient\_id, notification\_type, title, message, is\_read, related\_booking\_id?, related\_event\_id?, created\_at } .

### 3. Resources (rooms/labs/equipment)

| Method | Path                                           | Auth                                                                                      | Body                | Returns                                  |
|--------|------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------|------------------------------------------|
| GET    | /resources?<br>resource_type=&active_only=true | user                                                                                      | —                   | Resource[]                               |
| GET    | /resources/{id}                                | user                                                                                      | —                   | Resource                                 |
| POST   | /resources                                     |  admin | ResourceCreate      | Resource                                 |
| PATCH  | /resources/{id}                                |  admin | partial<br>Resource | Resource                                 |
| DELETE | /resources/{id}                                |  admin | —                   | 204 (soft-delet<br>→<br>is_active=false) |

**Resource:** { id, name, description?, resource\_type, location?, capacity?, requires\_approval, is\_active, created\_at } . **resource\_type** enum: classroom, lab, computer\_room, seminar\_hall, meeting\_room, equipment, other .

### 4. Availability (read-only; powers colours, search, free-slots)

| Method | Path                                                                                       | Auth |                        |
|--------|--------------------------------------------------------------------------------------------|------|------------------------|
| GET    | /availability/day?date=YYYY-MM-DD&resource_type=                                           | user | Resou                  |
| GET    | /availability/free-slots?<br>resource_id=&date=&duration_minutes=60&from_hour=8&to_hour=20 | user | { res<br>free_<br>[] } |

**ResourceAvailability:** { id, name, resource\_type, location?, capacity?, requires\_approval, is\_free, busy: Interval[] }. **Interval:** { start, end } (ISO datetimes).

## 5. Events

| Method | Path                               | Auth            | Body                                                                           | Return                          |
|--------|------------------------------------|-----------------|--------------------------------------------------------------------------------|---------------------------------|
| GET    | /events                            | user            | —                                                                              | Event[]                         |
| POST   | /events                            | user            | EventCreate                                                                    | Event (on event + its bookings) |
| POST   | /events/recurring                  | user            | RecurringEventCreate                                                           | { event_id, rule_id, .          |
| GET    | /events/calendar?<br>start=&end=   | user            | —                                                                              | CalendarB.<br>(RRULEs expanded) |
| GET    | /events/{id}                       | user            | —                                                                              | event detail<br>bookings        |
| PATCH  | /events/{id}                       | organizer/admin | { start_time?,<br>end_time?, title?,<br>description?,<br>occurrence_date? }    | updated                         |
| POST   | /events/{id}/cancel                | organizer/admin | { occurrence_date? }                                                           | cancels eve<br>one occur        |
| POST   | /events/{id}/cancel-<br>occurrence | organizer/admin | { occurrence_date }                                                            | cancel one<br>occurrence        |
| POST   | /events/{id}/edit-<br>occurrence   | organizer/admin | { occurrence_date,<br>new_start, new_end,<br>new_title?,<br>new_description? } | edit one<br>occurrence          |
|        |                                    |                 |                                                                                | delete who                      |

|        |                     |                 |   |             |
|--------|---------------------|-----------------|---|-------------|
| DELETE | /events/{id}/series | organizer/admin | — | recurring s |
|--------|---------------------|-----------------|---|-------------|

**EventCreate:** { title, description?, start\_time, end\_time, is\_public, bookings: BookingCreate[], group\_ids: string[], category }. **BookingCreate:** { resource\_id, start\_time, end\_time, notes? }. **category enum:** academic, adhoc. **Event:** { id, title, description?, organizer\_id, start\_time, end\_time, status, is\_public, created\_at }. **status (EventStatus):** draft, confirmed, cancelled. **RecurringEventCreate:** { title, description?, rrule, series\_start, series\_end\_date, duration\_minutes, resource\_id?, is\_public, notes?, group\_ids: string[] } (rrule is RFC-5545, e.g. FREQ=WEEKLY;BYDAY=MO,WE ).

**Conflict behaviour:** creating/editing a booking returns 409 if the room is taken (venue clash) or if the event's groups share students with another event at that time (student clash — hard block).

## 6. Bookings (a claim on a resource for an event)

| Method | Path                                                   | Auth                                                                                      | Body                                    | Return        |
|--------|--------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------|---------------|
| GET    | /bookings?<br>status=&resource_id=                     | user (admin<br>sees all)                                                                  | —                                       | BookingWithD  |
| PATCH  | /bookings/{id}/review?<br>new_status=approved rejected |  admin | —                                       | { id, status  |
| PATCH  | /bookings/{id}/cancel                                  | owner/admin                                                                               | —                                       | { id, status  |
| PATCH  | /bookings/{id}                                         | owner/admin                                                                               | {start_time?,<br>end_time?,<br>notes? } | updated booki |

**BookingWithDetails:** { id, event\_id, resource\_id, requester\_id, start\_time, end\_time, status, notes?, created\_at, resource\_name?, event\_title?, requester\_name? }. **status (BookingStatus):** pending, approved, confirmed, rejected, cancelled.

## 7. Groups & Roster (cohorts for clash detection)

| Method | Path    | Auth | Body | Returns                                                        |
|--------|---------|------|------|----------------------------------------------------------------|
| GET    | /groups | user | —    | { id, name,<br>description?,<br>group_type?,<br>member_count } |

|        |                                               |      |                                                  |                                                 |
|--------|-----------------------------------------------|------|--------------------------------------------------|-------------------------------------------------|
|        |                                               |      |                                                  | <code>[]</code>                                 |
| POST   | <code>/groups</code>                          | user | <code>{ name, description?, group_type? }</code> | group                                           |
| GET    | <code>/groups/{id}</code>                     | user | —                                                | group +<br><code>members: RosterPerson[]</code> |
| DELETE | <code>/groups/{id}</code>                     | user | —                                                | <code>204</code>                                |
| POST   | <code>/groups/{id}/members/{person_id}</code> | user | —                                                | <code>204</code> (add person to group)          |
| DELETE | <code>/groups/{id}/members/{person_id}</code> | user | —                                                | <code>204</code>                                |
| GET    | <code>/roster</code>                          | user | —                                                | <code>RosterPerson[]</code>                     |
| POST   | <code>/roster</code>                          | user | <code>{ full_name, email? }</code>               | <code>RosterPerson</code>                       |

**RosterPerson:** `{ id, full_name, email? }`.

## 8. Clash detection

| Method | Path                                             | Auth | Body / Query                                                           | Returns                                                |
|--------|--------------------------------------------------|------|------------------------------------------------------------------------|--------------------------------------------------------|
| POST   | <code>/clashes/preview</code>                    | user | <code>{ start_time, end_time, group_ids: [], resource_ids: [] }</code> | <code>ClashInfo[]</code>                               |
| GET    | <code>/clashes/event/{id}?start=&amp;end=</code> | user | (start/end optional → preview at a new time)                           | <code>ClashInfo[]</code><br>(student detail host-only) |

**ClashInfo:** `{ event_id, title, start, end, venue_clash, shared_resource_ids: [], venue_bookings: VenueBooking[], student_clash, shared_student_count }`. **VenueBooking:** `{ booking_id, resource_id, resource_name?, holder_id, holder_name? }` (who to send a release request to).

**Privacy:** only a count of shared students is ever returned — never names. Student-clash detail is hidden from non-hosts on the `event/{id}` endpoint.

## 9. Request-Release (negotiate a held slot)



| Method | Path                           | Auth         | Body          | Returns                               |
|--------|--------------------------------|--------------|---------------|---------------------------------------|
| POST   | /release-requests              | user         | ReleaseCreate | ReleaseRequest                        |
| GET    | /release-requests/incoming     | user         | —             | requests where I'm the holder         |
| GET    | /release-requests/outgoing     | user         | —             | requests I sent                       |
| POST   | /release-requests/{id}/accept  | holder/admin | ReleaseAccept | frees slot + fulfils requester intent |
| POST   | /release-requests/{id}/decline | holder/admin | —             | declines                              |
| POST   | /release-requests/{id}/cancel  | requester    | —             | withdraws                             |

**ReleaseCreate:** { booking\_id, message?, proposed\_event?: ProposedEvent }.

**ProposedEvent** (what to do on accept): { title?, description?, start\_time, end\_time, resource\_id?, group\_ids: [], category, move\_event\_id? } — if move\_event\_id is set, accept **moves that existing event** into the freed slot; otherwise it **creates** a new event from these details. (Omit proposed\_event entirely → accept just frees the slot.) **ReleaseAccept:** { mode: "cancel" | "shift", new\_start?, new\_end? } ( shift moves the holder's event to the new time). **ReleaseRequest:** { id, booking\_id, status, message?, response\_note?, created\_at, resolved\_at?, requester\_name?, holder\_name?, resource\_name?, event\_title?, start\_time?, end\_time? } . **status (ReleaseStatus):** requested, accepted\_released, accepted\_moved, declined, cancelled .

## 10. Feedback

| Method | Path      | Auth                                                                                      | Body                                                   |
|--------|-----------|-------------------------------------------------------------------------------------------|--------------------------------------------------------|
| POST   | /feedback | user                                                                                      | { message, category, page_url?, page_name?, browser? } |
| GET    | /feedback |  admin | —                                                      |

## Demo accounts (seeded)

admin@rsp.edu / admin123 (admin), alice@rsp.edu / alice123 (professor), bob@rsp.edu / bob123 (professor). carol@rsp.edu / carol123 (staff).

# How to prompt Claude (design) so integration is painless

Goal: get a UI whose data calls already match these endpoints, so wiring it up later = drop in the base URL + token, not reshape everything.

1. Give it the contract. Attach `openapi.json` (or paste this file). Tell it:

"This is my real backend's OpenAPI spec. Build the UI to talk to **these exact endpoints and field names** — do not invent endpoints or rename fields."

2. State the hard rules (paste verbatim):

- All calls go to a base URL `/api/v1`.
- Auth: `POST /auth/login {email,password}` returns a JWT; send `Authorization: Bearer <token>` on every other call; store the token; redirect to login on `401`.
- Datetimes are ISO-8601 UTC strings; IDs are strings; enums are exactly the values in the spec.
- On `409`, show the `detail` message (it's a real conflict like "room taken" or "student clash").

3. Ask for a thin, swappable API layer:

"Put **all** network calls in one `api` module (e.g. an axios instance with the base URL + a Bearer interceptor) and one function per endpoint, named after it. Components call those functions only — no inline fetch, no data reshaping."

This is the key to easy integration: the new UI's `api.js` becomes a near-drop-in replacement.

4. Describe the core flows it must model (so screens map to real endpoints):

- **Login** → store token.
- **Resources page**: `GET /resources` + `GET /availability/day?date=` for free/busy dots + a free-slot finder (`GET /availability/free-slots`).
- **Create event**: `POST /clashes/preview` live as the user edits, then `POST /events`. On a venue clash, offer Request-Release (`POST /release-requests` with `proposed_event`).
- **Groups & roster**: the `/groups` + `/roster` endpoints.
- **Requests dashboard**: `/release-requests/incoming|outgoing`, accept with `{mode: 'cancel'|'shift'}`.
- **Calendar**: `GET /events/calendar?start=&end=`; edits via `PATCH /events/{id}`.

5. Tell it to flag gaps, not invent them:

"If a screen needs data this API doesn't expose, **list it as a 'backend gap'** instead of assuming a new endpoint. I'll add those endpoints."

**6. Match the existing client (optional but ideal).** This app's current client is `frontend/src/lib/api.js` (axios + JWT interceptor) and Zustand for the auth store. Asking Claude to follow that same pattern means the new UI can reuse the exact same `api.js`.

**Integration checklist when you bring the new UI in:**

- Point its API base at `/api/v1` (keep the Vite proxy, or set `VITE_API_URL`).
- Confirm it reads the JWT exactly as login returns it.
- Diff its endpoint calls against `openapi.json` — any mismatch is a quick rename.
- Anything in its "backend gaps" list → tell me and I'll add the endpoint